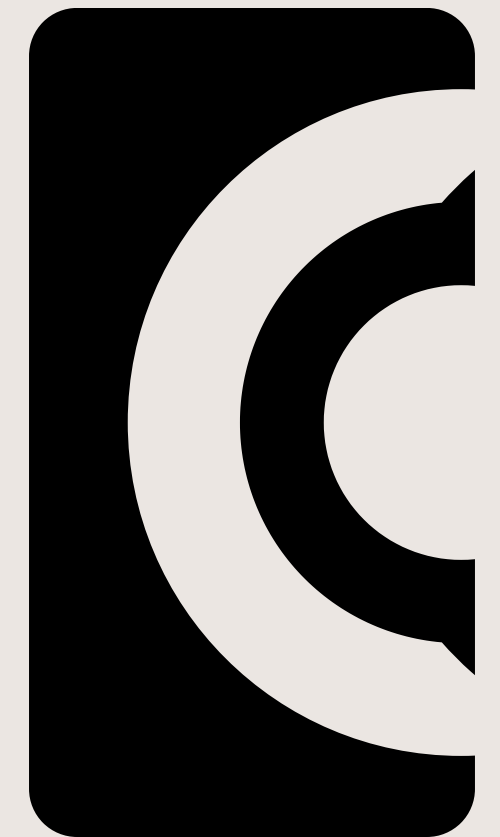


EasySpot

Parking Analytics Platform



Agenda

Product Owner

Architecture

Deployment

Agile Process

Live Demo

Final Reflection

Product Owner

EasySpot, is a smart parking management platform that provides real-time location and monitoring of parking spaces. It offers a platform across web and mobile interfaces for all users: a driver experience to find, filter, and navigate to available spots, and a data dashboard for operators. The system integrates IoT sensor APIs to capture occupancy data, generate usage statistics, and provide predictive insights.

Personas

who suffers most when a parking lot is poorly managed

- the **Manager** loses revenue and has no data to act on;
- the **Technician** is reactive instead of proactive because faults go unnoticed;
- the **Driver** wastes time circling for a spot or dealing with broken payment systems



 regular driver



 Mobility-Impaired Driver



 EV Driver



 Parking Manager



 Maintenance Technician

Main User Stories

- **User Story #1: Real-time Parking Availability**

As a **regular driver**,

I want to check the number of available parking spots in advance, so that I can minimize my search time and ensure I arrive at my meetings on time.

- **User Story #13: Infrastructure Mapping & Status Monitoring**

As a **field maintenance technician**,

I want to see a live map of all hardware components and their individual status, so that I can perform proactive maintenance and ensure the parking ecosystem remains fully operational for all users.

- **User Story #14: Repair & Update of sensors**

As a **field maintenance technician**,

I want to be able to update the state of the sensors when I repair a malfunction. So that the system accurately reflects their current operational condition.

- **User Story #8: Accessible Spot Filtering & Proximity**

As **Mobility-Impaired Driver**,

I want to filter the parking spots for the specifically designated accessible and see the exact distance to the parking entrance, so that I can ensure the walk the least distance to my medical appointment is manageable and within my physical limits.

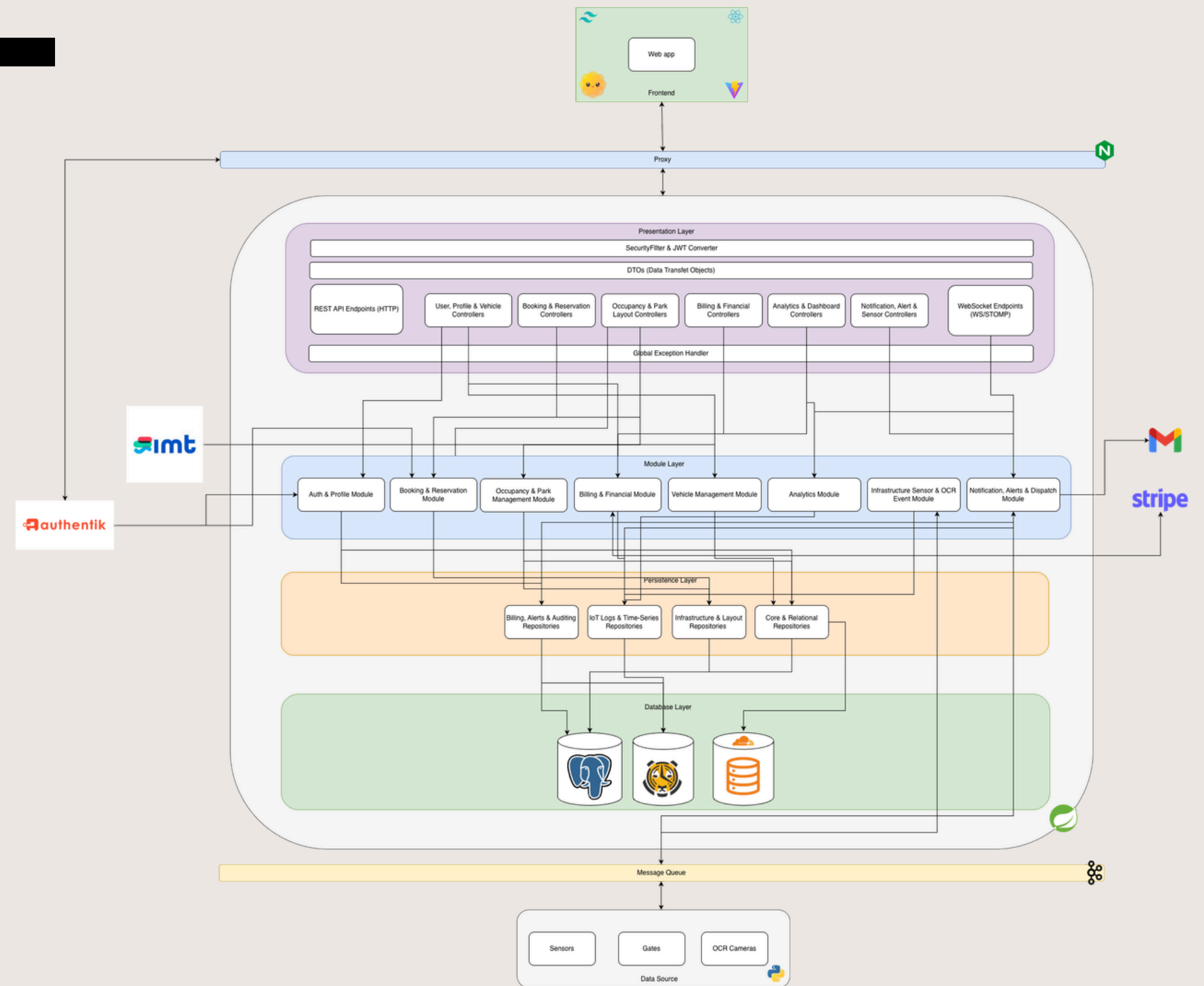
- **User Story #9: Reliability & Physical Compliance**

As a **Mobility-Impaired driver**,

I want to receive real-time confirmation of spot availability and space dimensions, so that I don't risk arriving at a spot that is either illegally occupied by an unauthorized vehicle or too narrow for me to safely deploy my wheelchair.

Architec- ture

Our architecture is organized into independent modules within the module layer. Each module communicates directly with its dedicated repositories in the persistence layer, while also supporting inter-module communication to orchestrate complex business workflows across domains such as booking, billing, occupancy, vehicle management, analytics, and notifications.



Deployment

☐		authentik-red	1585c1112270	redis:7.2-al	0.13%	13.02MB / 7.75G	0.16%	11M			
☐		kafka-1	b5128d2885be	apache/kaf 9092:9092 Show all ports (2)	2.91%	733.3MB / 7.75G	9.24%	212			
☐		authentik-boc	54654ad20b8f	python:3.1	0%	0B / 0B	0%	0B /			
☐		authentik-wo	eadd9e179427	goauthenti	0.03%	325.9MB / 7.75G	4.11%	6.86			
☐		authentik-ser	5b99586a83f7	goauthenti 9000:9000	0.24%	465.4MB / 7.75G	5.86%	24.1			
☐		postgres-1	4f9e72449b49	postgres:1 5432:5432	42.64%	59.98MB / 7.75G	0.76%	37.3			
☐		timescaledb-	5562a11ee074	timescale/ 5433:5432	156.31%	395.4MB / 7.75G	4.98%	245			
☐		nginx-1	2699e49e9c0b	nginx:1.27 80:80	0%	2.3MB / 7.75GB	0.03%	0B /			
☐		frontend-1	ca6bd48c6fd3	projx-fronte 5173:5173	1.6%	124.2MB / 7.75G	1.57%	156			
☐		scraper-vehic	4c9c517871e8	projx-scrap	0%	76.09MB / 7.75G	0.96%	0B /			
☐		api-1	fc75118e9740	projx-api 8080:8080	38.06%	553.2MB / 7.75G	6.97%	0B /			
☐		sensors-1	8394b65f3ef5	projx-sens	1.03%	49.47MB / 7.75G	0.62%	0B /			

Pre-commit checks (PRs)

on: pull_request

1

✓ pre-commit

SonarQube Analysis

on: pull_request

CI - repository validation

on: pull_request

6

✓ Lint configs (YAML / JSON / Dockerfile)

✓ Backend tests (unit + integration)

✓ Sensors tests (Python)

✓ Backend Newman validation

✓ Frontend tests (build + unit/integration)

✓ Frontend E2E (Playwright)

Agile Process Backlog

detiuaveiro / Projects / G703 IES

Q Type / to search

+

G703 IES

Add status update

Insights

Workflows 10

Backlog

Priority board

Team items

Roadmap

My items

New view

Filter by keyword or by field

View

Title	Assignees	Status	Linked pull requests	Sub-issues progress
33 To avoid showing all completed tasks, filter by weekly schedule; #194	inessgomes	Done		
34 chore: configure repository — CI, templates, linting, gitignore, and gitflow #9	dino3111 and marti...	Done	#10	
35 Documentation: Write Minutes for Meeting #1 #16	mariamane73	Done	#11	
36 Documentation: Write Minutes for Meeting #2 #17	mariamane73	Done	#11	
37 docs: Define and Document System Architecture #55	martingil	Done	#69	5 / 5 100%
38 docs: Write Minutes for Meeting #3 #41	mariamane73	Done	#59	
39 docs: Write Minutes for Meeting #4 #60	mariamane73	Done	#59	
40 Implement endpoint POST /api/account/type (User Role Assignment) #77	inessgomes	Done		
41 Implement endpoint POST /api/driver/type (Driver Subtype Selection) #78	inessgomes	Done		
42 Implement endpoint POST /api/parks/{id}/favorite (Toggle Favorite Parking Lot) #84	martingil	Done		
43 Implement endpoint GET /api/driver/costs/spending (Driver Spending Analytics) #86	mariamane73	Done	#125	
44 Implement endpoints GET & PUT /api/profile (Role-Aware User Profile) #89	martingil	Done		
45 Implement endpoint POST /api/alerts (Availability Alerts + Email Summary) #92	mariamane73	Done		

+

You can use Control + Space to add an item

Agile Process

Pull requests + changelog por tag

126 Pull requests

changelog por tag

- api - 54
- architecture - 19
- bug - 30
- code quality - 3
- devops - 3
- documentação - 15
- enhancement - 64
- frontend - 26
- workflows - 3

Agile Process

resumo de contribuições

Membro	nº commits	nº pull requests submetidos	nº pull requests aprovados	nº de participações em revisões de PRs
Martim	365	54	54	40
Inês	58	25	25	7
Claudino	43	29	29	11
Maria	40	26	26	11

Live Demo

Additions

Since 14

- **sensors;**
- **kafka;**
- **websockets.**

- Introduced fault simulation (NetworkFaultInjector, SpotStateMachine) to mimic real-world failures and recoveries. Implemented SpotStateResolver for clean status transitions. Improved frontend visibility with floor filtering and sorted sensor display. Fixed several bugs in fault integration and thread safety.
- Reorganized consumers by domain (gate, sensor, OCR). Added alert state and backend decision history tracking for auditability.
- Extended real-time updates to technician and maintenance pages with proper Spring Security and authentication integration.

Problems

CI/CD Instability

Problem: Pipeline failures caused by missing environment variables, Authentik configuration not replicated in test environments, and linting rules only enforced in CI.

Solution: The Authentik integration required a bootstrap script to provision the initial configuration, but this script was not consistently maintained or documented, meaning new environments would fail silently. The solution is to make the bootstrap script idempotent and part of the standard setup process, with all required Authentik properties explicitly documented and versioned alongside the codebase. For linting, aligning local and CI rules early avoids a class of avoidable pipeline failures.

Tests Written After the Fact and Outdated Mocks

Problem: Tests written after the code was already done, mocks becoming outdated when APIs changed, and flaky E2E tests.

Solution: Adopt TDD, or at minimum write tests in parallel with the code in the same PR. For mocks, use a shared API contract (e.g. OpenAPI spec) as the single source of truth, so that when an API changes the mock is regenerated automatically. For E2E tests, isolate each test with its own data instead of relying on a shared global seed.

Underestimated Complexity of the Billing/Gate System

Problem: Billing edge cases (dailyMax cap, walk-in sessions, idempotency) only discovered late, during integration.

Solution: During the design phase, use event storming or explicitly map out critical business flows (entry, exit, payment failure) before writing any code. Define the contract between the gate, billing, and sensor subsystems with integration tests from the start, not at the end. Business edge cases should be captured as tickets early on, not discovered during code review.

Final Reflexion

thank you

EasySpot

Claudino Martins 127368
Inês Lourenço 125895
Maria Mané 125102
Martim Gil 124833